Sprite Instructions

The game is **fully playable without any imported sprites** — `Assets/Scripts/Core/SpriteFactory.cs`
generates simple coloured shapes at runtime (triangles for ships, circles for bullets/power-ups, squares for
some enemies, soft dots for stars). This document describes the recommended custom sprites if you want to
replace the procedural placeholders with real art.

Import settings (apply to all sprites)

- **Texture Type**: `Sprite (2D and UI)`
- **Pixels Per Unit**: `100` (matches `SpriteFactory.PixelsPerUnit`)
- **Filter Mode**: `Bilinear` (or `Point` for pixel-art)
- **Compression**: `None` or `High Quality`
- **Format**: PNG with transparency (RGBA32)
- **Pivot**: `Center`

Recommended sprite list

Purpose	Suggested file name	Dimensions (px)	Notes
Player ship	`player_ship.png`	64 × 64	Points **up**. Cool blue palette.
Basic enemy	`enemy_basic.png`	64 × 64	Points **down**. Red palette.
Zigzag enemy	`enemy_zigzag.png`	48 × 48	Orange/amber.
Circular enemy	`enemy_circular.png`	48 × 48	Purple, round.
Boss	`enemy_boss.png`	128 × 128	Large, menacing red.
Player bullet	`bullet_player.png`	24 × 24	Bright cyan/white, can be elongated (art points up).
Enemy bullet	`bullet_enemy.png`	24 × 24	Red/pink.
Power-up: Health	`powerup_health.png`	32 × 32	Green cross/heart.
Power-up: Shield	`powerup_shield.png`	32 × 32	Blue shield.
Power-up: RapidFire	`powerup_rapidfire.png`	32 × 32	Yellow lightning.
Power-up: Spread	`powerup_spread.png`	32 × 32	Orange tri-arrow.
Power-up: x2 Score	`powerup_score.png`	32 × 32	Magenta star/“x2”.
Shield bubble FX	`shield_bubble.png`	64 × 64	Soft translucent ring around the player.
Star (parallax)	`star.png`	8 × 8	Soft white dot.
Explosion frames	`explosion_##.png`	64 × 64 each	Optional sprite-sheet if replacing the particle FX.

How to use custom sprites

The cleanest integration is to load your sprites and assign them where the entities currently call `SpriteFactory`:

- **Player** — in `PlayerController.Initialize`, replace `SpriteFactory.CreateShipSprite(...)` with your loaded `player_ship` sprite.

- **Enemies** — in `Enemy.ApplyAppearance`, swap the `SpriteFactory.*` calls per `EnemyType`.
- **Bullets** — in `Bullet.Launch`, replace `SpriteFactory.CreateCircleSprite(...)`.
- **Power-ups** — in `PowerUp.Configure`, replace `SpriteFactory.CreateCircleSprite(...)`.

A convenient pattern is to put your PNGs under `Assets/Resources/Sprites/` and load them with `Resources.Load<Sprite>("Sprites/player_ship")`, then cache the result. Alternatively, refactor the entities
to expose `public Sprite` fields and assign them in prefabs.

> Tip: keep the same approximate proportions and pivots as the table above so collision radii (set in code)
> still line up with the visuals. If your art is a different size, adjust the *`transform.localScale`* / collider radius in the corresponding *`Configure`* / *`Initialize`* method.